RT Core Microarchitecture study using Vulkan-Sim

anaitha2@ncsu.edu

Important concepts/terms in Ray tracing

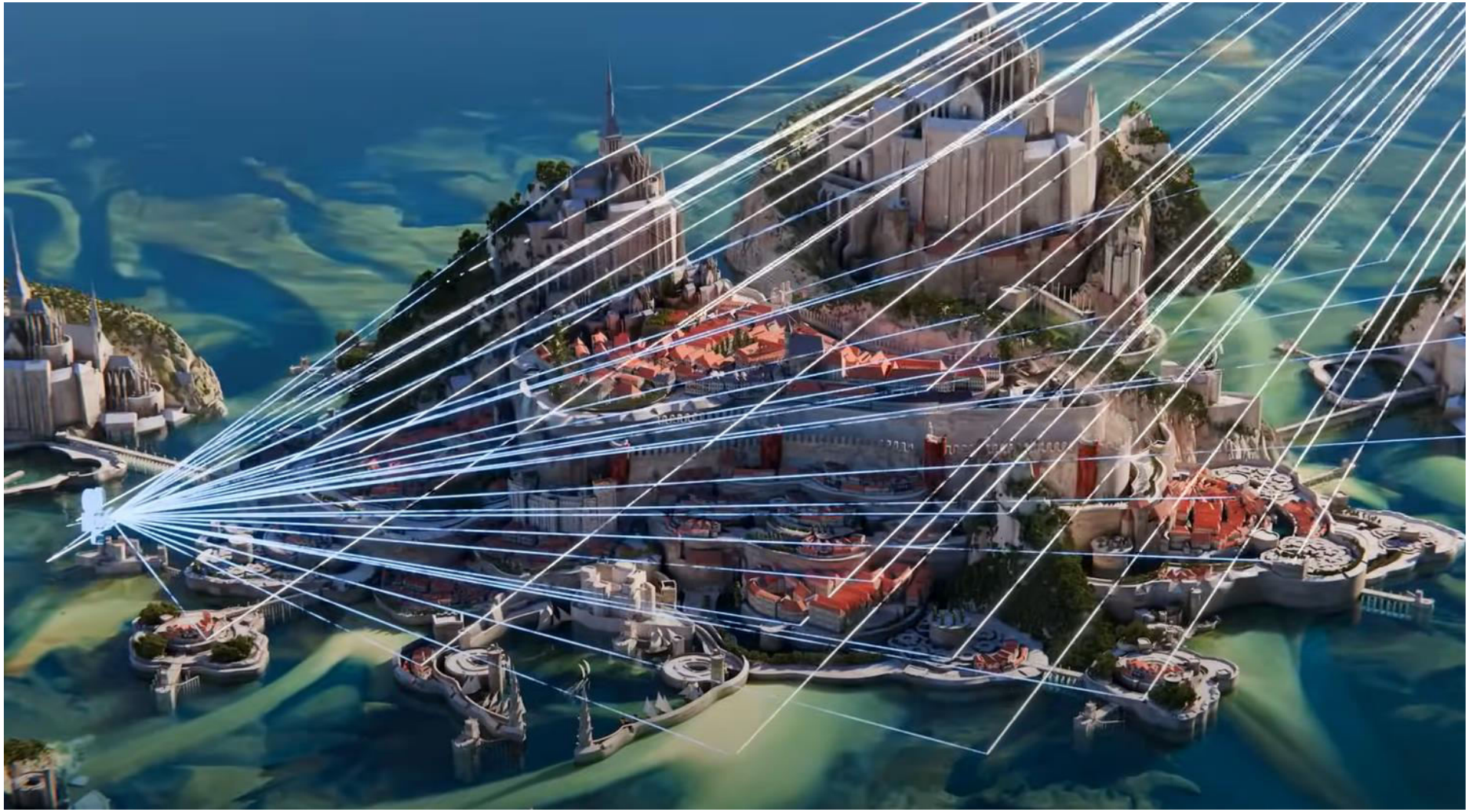
- Triangle Mesh
- Procedural Mesh
- Bounding Volumne Hierarchy (BVH)
- Top level Accelerating Structure (TLAS)
- Bottom level Accelerating Structure (TLAS)
- Intersection shader
- Anyhit shader
- Closest hit shader
- Miss shader

Triangle and Procedural mesh

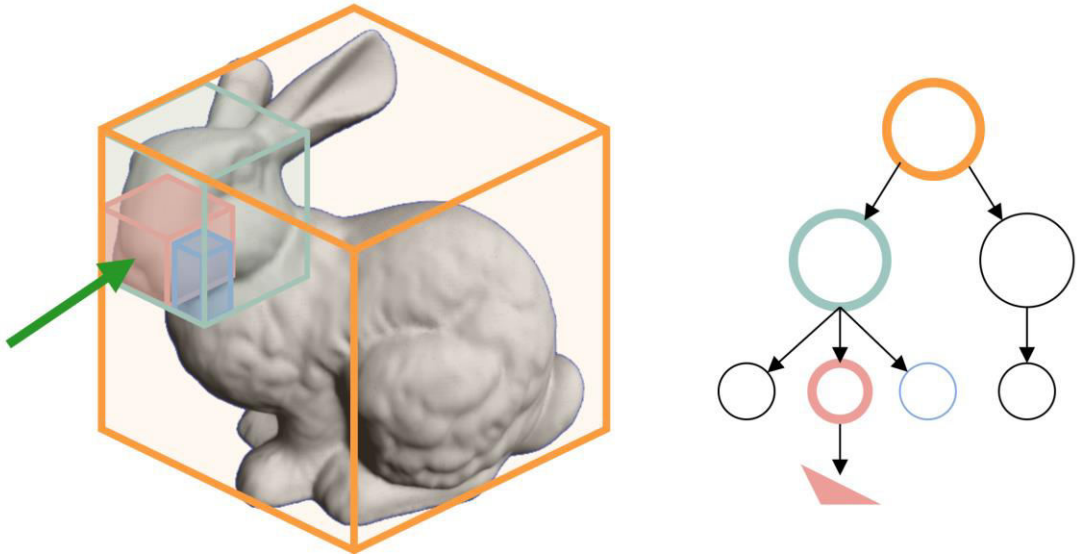
- A triangle mesh is a 3D object represented as a collection of triangles (or polygons) that define its surface.
- Triangle meshes are static and typically do not change their shape during runtime.
- Triangle meshes store explicit vertex positions, normals, UV coordinates, and other attributes for each vertex, which can consume a significant amount of memory.
- Rendering pre-defined triangle meshes tends to be faster, as the data is already prepared.
- Used for static 3D objects like characters and buildings.

- A procedural mesh is generated algorithmically at runtime. Instead of defining the mesh manually, you specify a set of rules and parameters to create the mesh procedurally.
- Procedural meshes are highly flexible and can be modified in real-time.You can create complex shapes or deformations by changing parameters or applying algorithms.
- Procedural meshes can be memory-efficient because they often generate vertex data on-the-fly based on algorithms, reducing memory consumption.
- Generating procedural meshes in real-time can be computationally intensive, and rendering them may require more processing power.
- Used for dynamic and adaptive objects like terrain generation and deformable surfaces.

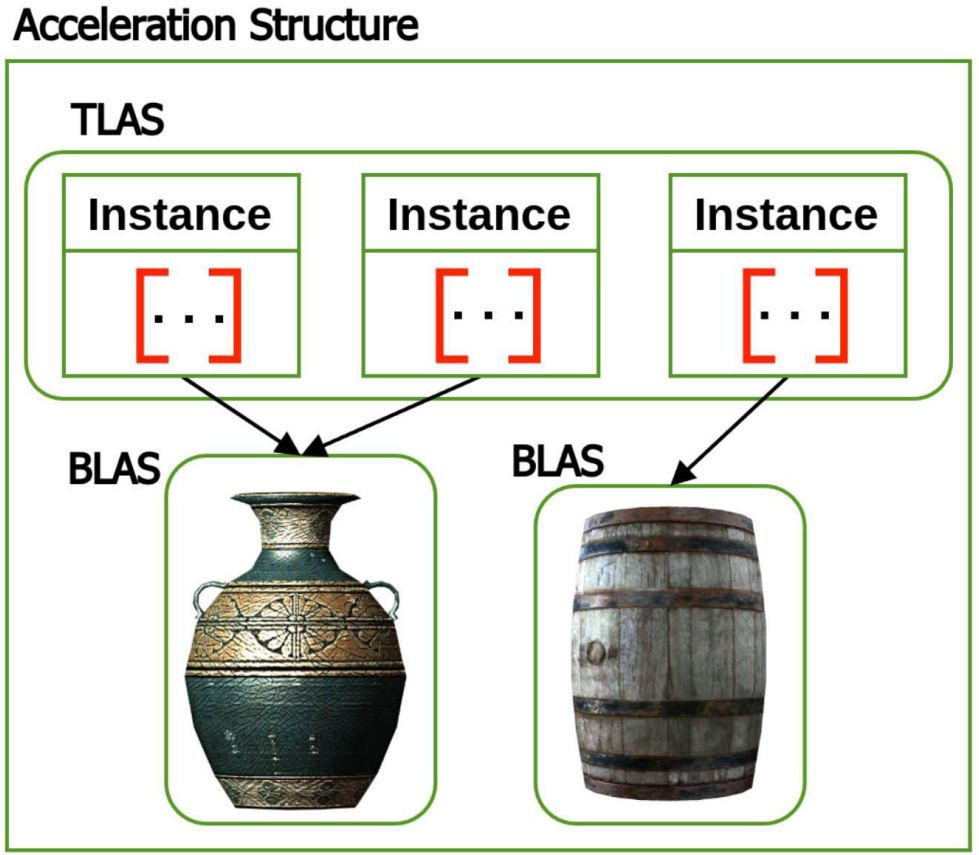
Ray Tracing



Ray Tracing Data Structures: BVH & AS



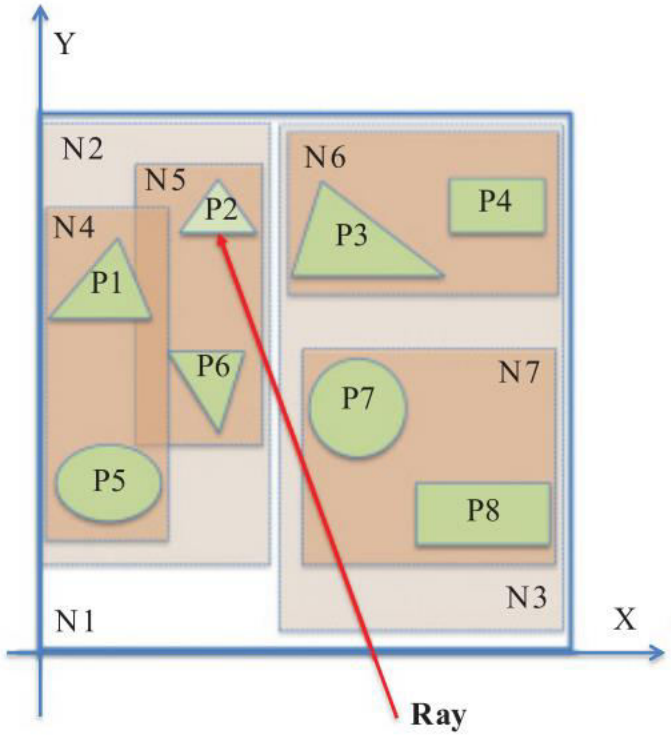
BVH

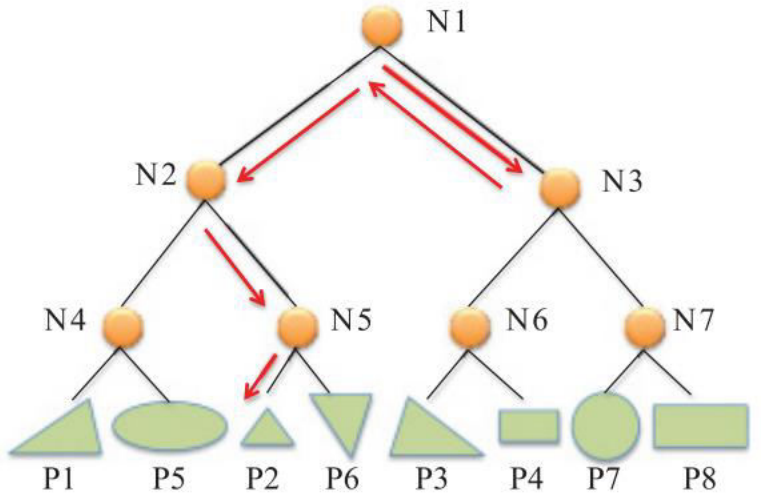


Ray tracing often uses a two-level BVH structure (TLAS and BLAS) to accelerate ray-object intersection tests. The TLAS holds instances of objects, each referencing a BLAS, which contains the actual geometry. This hierarchical organization helps to efficiently traverse the scene and identify potential intersection candidates, improving performance.

Bounding Volumne Hierarchy (BVH)

- A tree structure of the environment/scene with each node associated with a bounding box called Axis Aligned Bounding Box (AABB)
- Two nodes on the same level can have their AABBs overlapped
- Nodes at higher level (here N1, N2, N3) are called Top Level Acceleration Structure (TLAS)
- Nodes at lower level are called Bottom Level Acceleration Structure (BLAS)
- The coordinate system at TLAS level is based on relative the ‘world’ while at the BLAS level it’s based on relative to the ‘object’
- So while traversing BVH, when you reach BLAS level you need to apply worldToObject transformation to the Ray





Intersection Shader and Anyhit Shader

- Intersection Shader is responsible for determining if a ray intersects with a specific geometry or primitive in a scene. When a ray is hit on an object, it also determines the point of intersection with geometry which is used for shading.

- Any Hit Shaders are invoked during ray traversal when a ray intersects with a surface. They are used for custom material-specific calculations. In Ray tracing they are specially used to determine the object’s transparency/translucency (whether the ray should pass through the object or not based on opaque property of object)

Closest hit shader and Miss shader

- Closest Hit Shader is invoked after a ray intersects with a surface, and it is responsible for performing shading calculations at the point of intersection. In other words, it determines how light interacts with the surface, calculates the material properties, and computes the final color of the pixel for that intersection.
- For a ray hitting a reflective surface, it would compute the reflection direction and color. For a refractive surface, it would handle refraction and possibly total internal reflection.
- Miss Shader is called when a ray does not hit any surface in the scene. It is responsible for specifying what color should be returned in such cases. Miss Shaders handle rays that "miss" all geometry in the scene and contribute to the background or environment color.
- They are used to render a clear sky, stars, or other background elements, or they can be used to set a specific color or texture for the background.

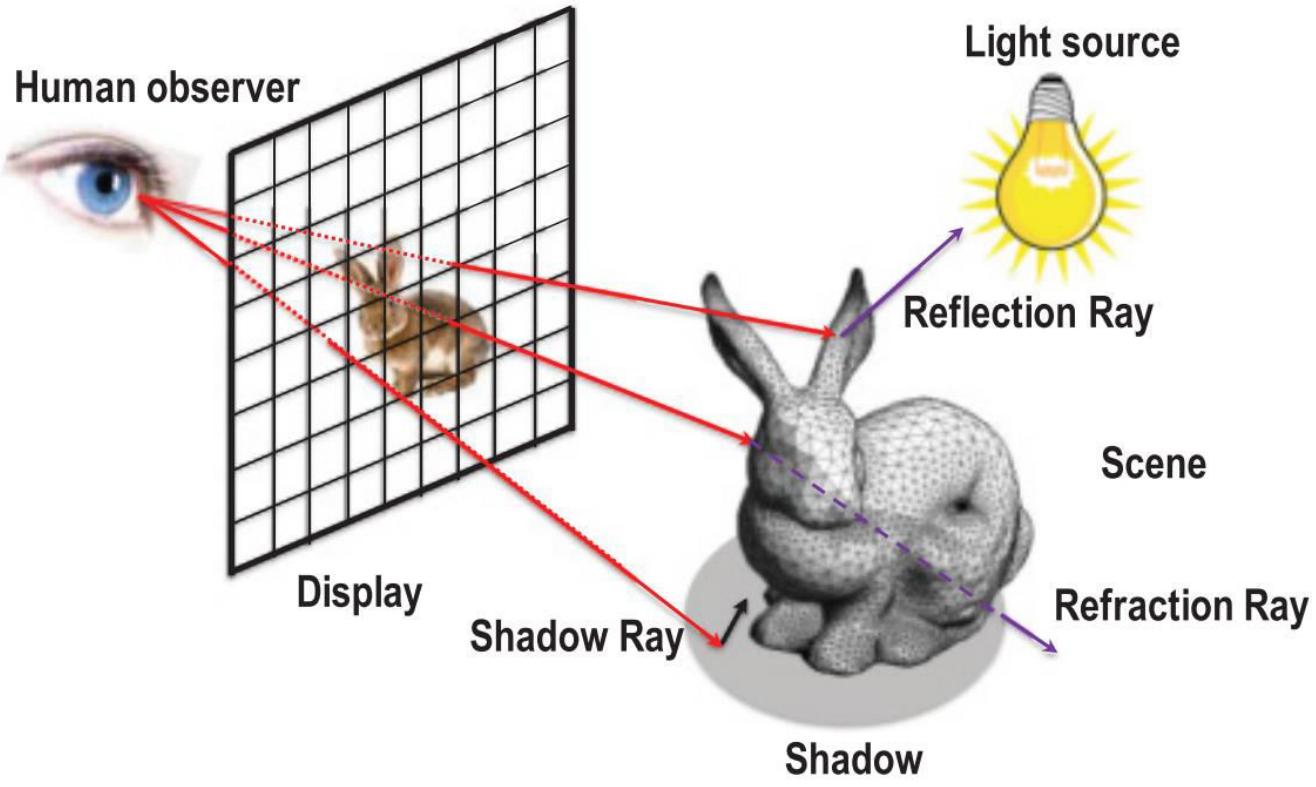
Some common rendering types in Ray tracing

With Primary rays (only involves rays origionated from camera to the object)

- Traditional rendering

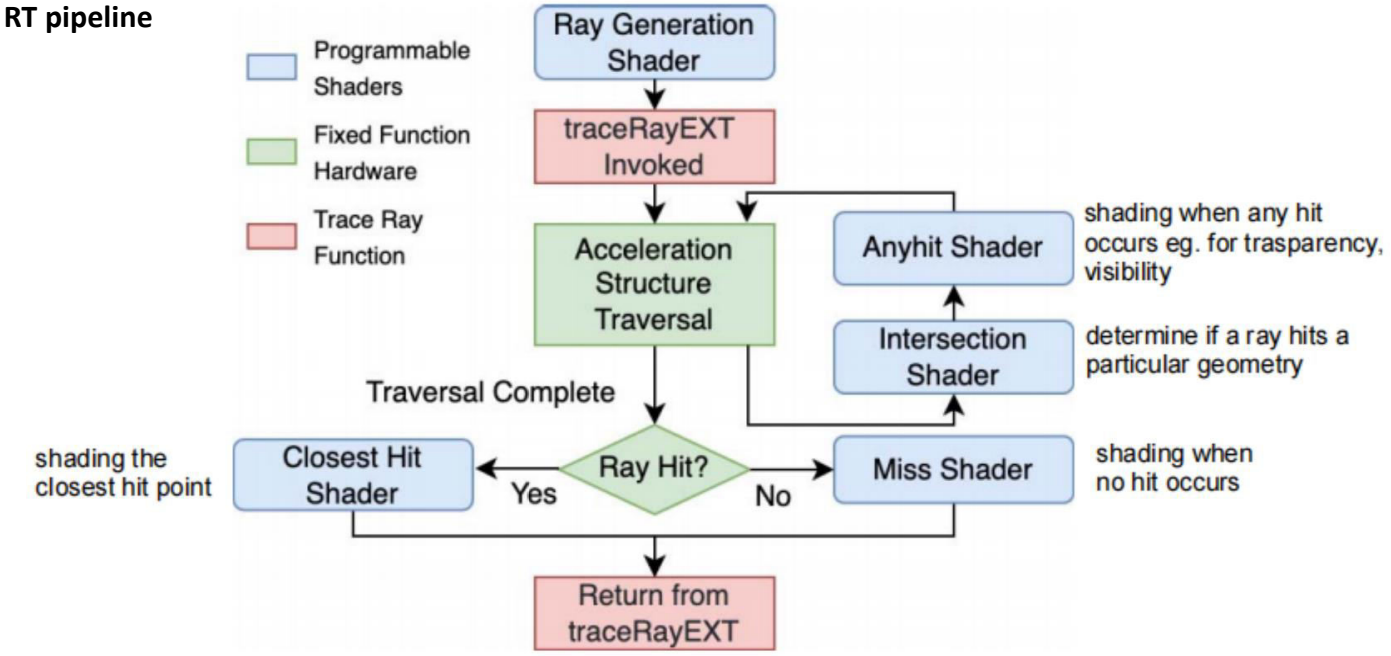
With Seconday rays (involves generation of secondary rays from hit object/geometry/surface to another direction)

- Shadows (secondary ray to light source)
- Reflections (secondary ray direction based on angle of incidence and angle of reflection)
- Refractions (secondary ray direction based on Snell's law angle of refraction)
- Ambient Occulution (secondary rays from the intersection point along every direction within a hemisphere)



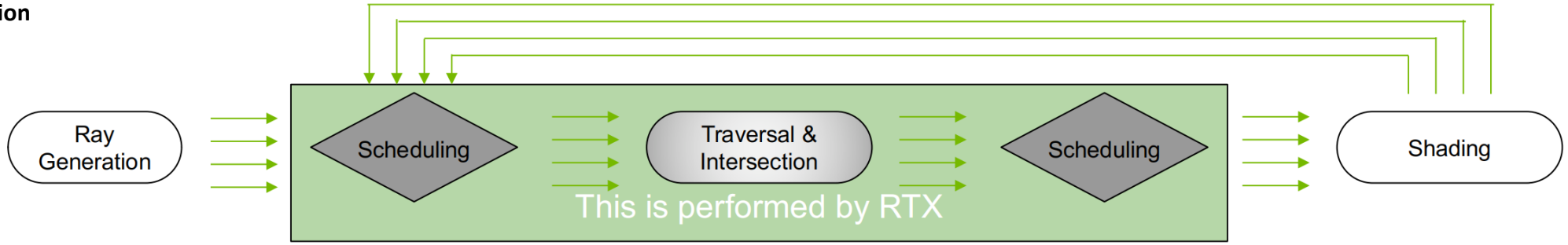
Vulkan Ray Tracing Pipeline vs Execution of shaders on actual hardware

- **Vulkan RT pipeline**



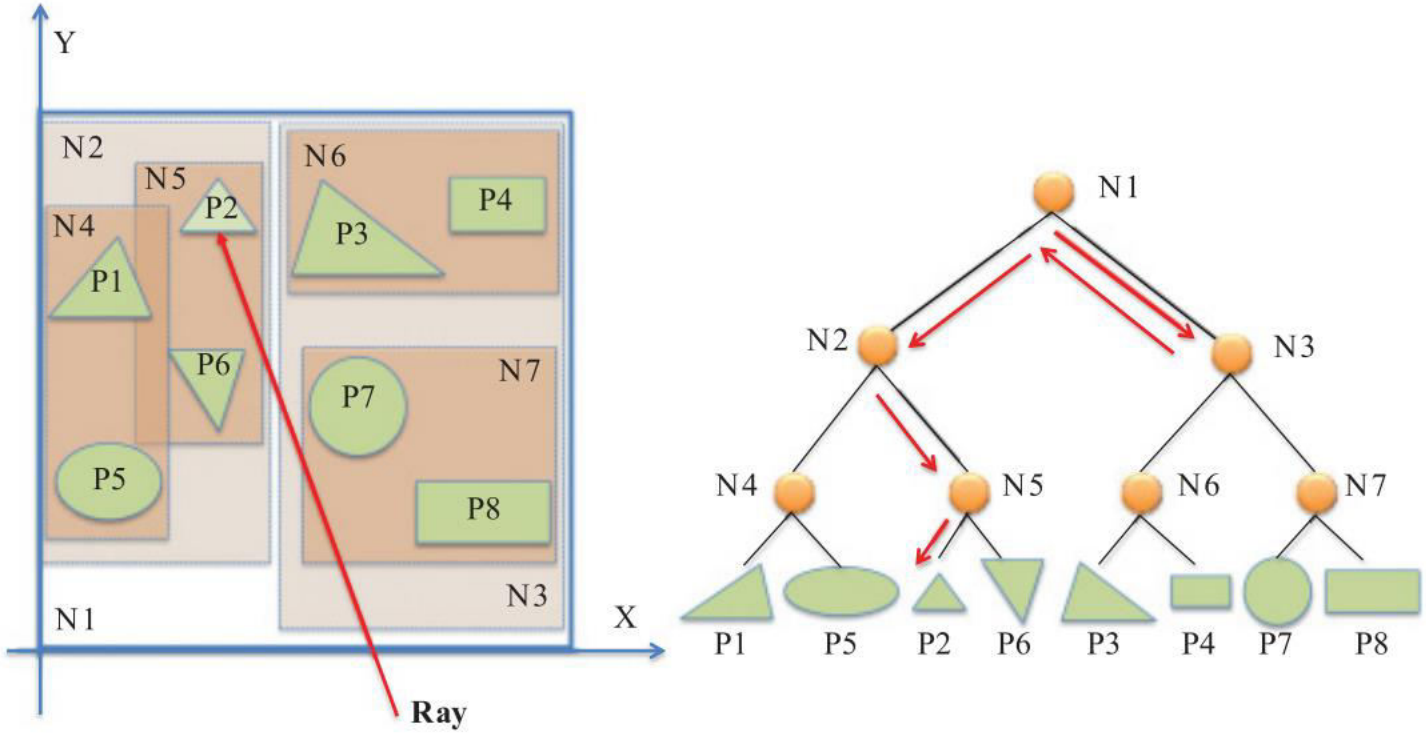
- In Vulkan RT pipeline the ‘Acceleration Structure Traversal’ and ‘Intersection Shader’ (which are defined for a triangle mesh) are executed in the RT core.
- All other shaders are executed in CUDA cores.
- Intersection shaders which are defined for procedural mesh are executed in CUDA cores as well.

- **Actual RT execution**

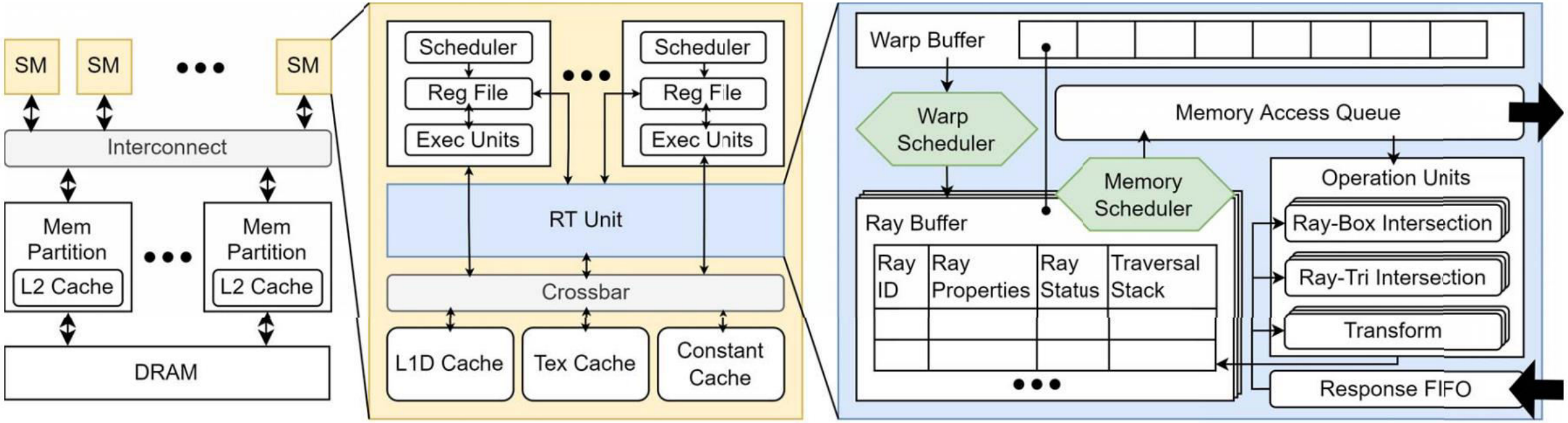


AS (BVH Tree) Traversal

- Depth first search



GPU microarchitecture implemented in Vulkan-Sim

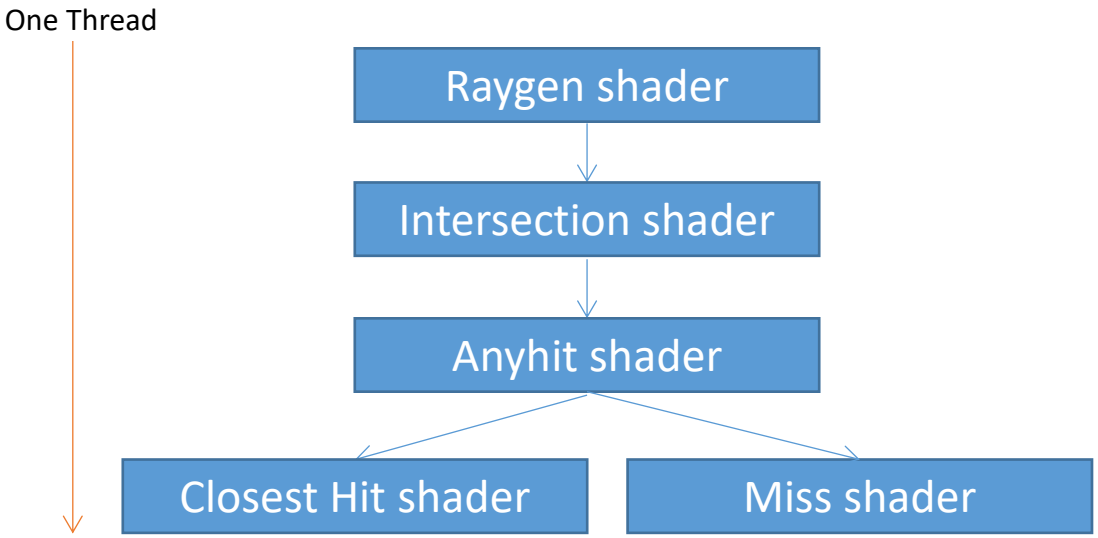


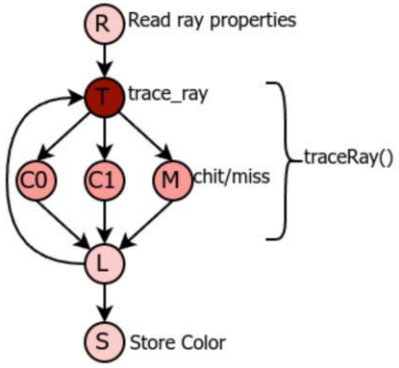
Execution Policies of the RT core microarchitecture

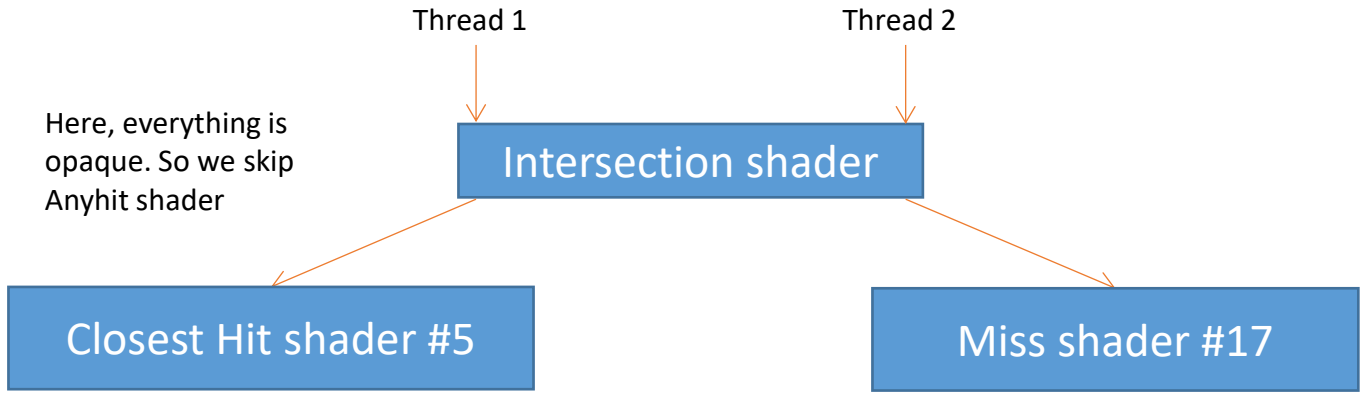
- Warp Size = 32 threads
  - Each Warp handles 32 pixels horizonally
- Branch divergence handling policy - Immediate Post dominator reconvergence (IPDOM)
- Warp scheduling policy - Greedy-then-Oldest (GTO)
  - The current warp keeps executing until it encounters a cache miss and stalls for memory request

One thread per raygen shader

- Each thread executes a raygen shader and continues to traverse the BVH tree and call other shaders sequentially when a traceRayEXT function is encountered
- Any thread divergence in the raygen shader continues in other shaders.

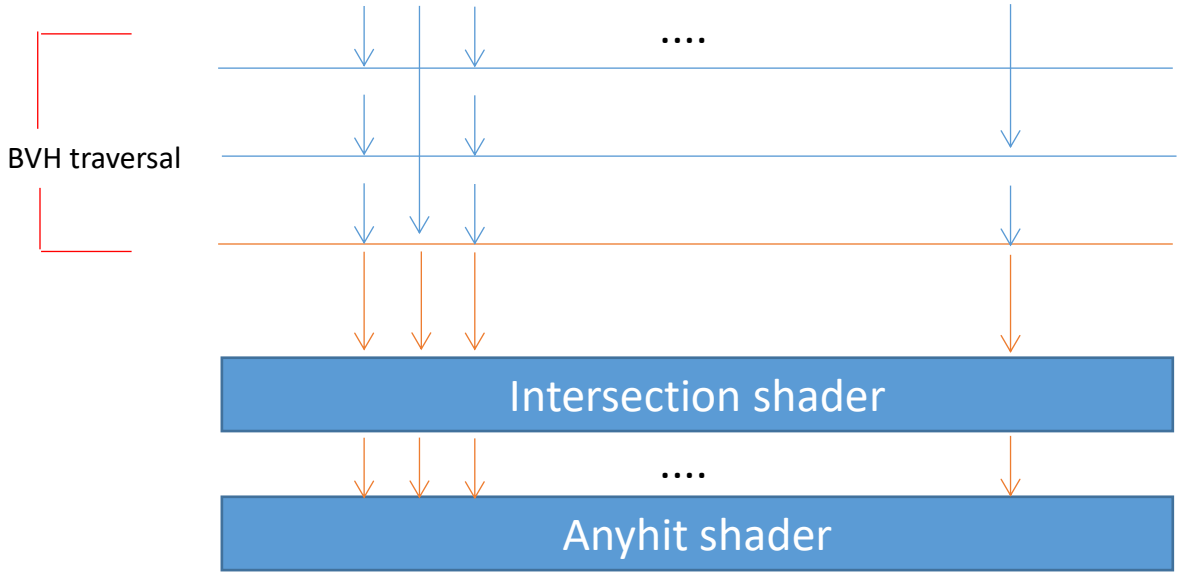






Delayed Intersection and Any-hit execution

- Two threads can execute very different set of shaders.
- Executes Intersection and Any-hit shaders after the traversal of all threads in the warp is completed
- Each thread executes all Any-hit and Intersection shaders for the ray sequentially
- Adds memory overhead - a list of Intersection and Any-hit shaders need to be stored during traversal for later execution



Even if any 1 ray is stalled, the entire warp is stalled

Warp Scheduling (Greedy-then-oldest)

Warp ID	cycle																					
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
W0	TLAS level 1	TLAS level 2	TLAS leaf	BLAS level 1											BLAS level 1	BLAS leaf	BLAS leaf	BLAS leaf						
W1					TLAS level 1	TLAS level 2	TLAS leaf	BLAS level 1	BLAS leaf										BLAS leaf	BLAS leaf	BLAS leaf			
W2										TLAS level 1	TLAS level 2	TLAS leaf	BLAS level 1	BLAS leaf								BLAS leaf	BLAS leaf	BLAS leaf

Details of the RT core microarchitecture

- RT core is a specialized Execution Unit for execution of Ray instructions
- 1 RT core per SM
- In Vulkan-Sim a total of 8 warps can exist in RT core
- Ray Buffer tracks ray properties, current status, and traversal stack
  - Ray Properties - Origin, direction, and t-parameters (light ray distance)
  - Traversal stack - contains unvisited nodes of BVH. 8 stack entries for every ray.
  - Ray/Warp status - Number of active threads in a warp

Memory Scheduler

- Memory scheduler checks
  - Ray status to check all threads which are ready to issue a memory request
  - If a thread is ready then get the address to be fetched from the Traversal stack
- The Memory scheduler
  - Collects all addresses to be fetched in a warp
  - Merges memory requests
  - Sends the collection of memory requests to Memory Access Queue
- All data from L1 cache is returned to Response FIFO
- On a cache miss, the memory request goes down memory hierarchy and eventually returns to the Response FIFO

Operations Unit

- Operations scheduler checks
  - Ray status - To check what threads are active and awaiting current memory response
  - Top entry of Traversal stack - To identify what BVH operation needs to be performed
- Operations Unit has 3 pipelined hardware units to perform different BVH operations
  - Ray-Box intersection
  - Ray-Triangle intersection
  - Coordinate transformation
- Once BVH operation is complete Operation unit updates Ray status
- On a primitive hit, the results are stored in memory and read back during the closest hit shader execution

PTX instructions for Ray tracing

- Graphics shaders are written in the OpenGL Shading Language (GLSL)
- These shaders are then compiled to an intermediate representation used exclusively by Mesa called NIR
- PTX code needs to be translated from NIR
- Some Ray tracing instructions in NIR can directly be translated to PTX instructions
- Other Ray tracing instructions instructions need custom PTX instructions in order to be translated to low-level instructions
- Vulkan-Sim introduces some custom PTX instructions for Ray tracing

Table II: Notable Vulkan-Sim Custom PTX Instructions

PTX Instruction	**Instruction Description**
traverseAS	Traverse the acceleration structure
endTraceRay	Pop traversal results stack and clear intersection table
rt_alloc_mem	Allocate memory and load address for variables shared among different shaders
load_ray_launch_id	Load a unique ray ID for each thread

Execution performed by:
CPU
CUDA Core
RT Core

Ray Tracing Pipeline for traditional Ray Tracing for

1. Construction of Bounding Volumne Hierarchy (BVH)
2. Generation of Primary ray
3. Traverse the BVH structure till TLAS leaf nodes
4. Perform Intersection tests on BLAS leaf nodes
    1. Triangle Mesh
    2. Procedural Mesh
5. Process hit information and create Ray payload
6. Apply Closest-hit shader or Miss shader based on information in Ray payload

Ray Generation (CUDA Core)

- The ray is created for the specific thread. In our run in Vulkansim we create 1 ray per pixel.
- The ray is defined by specifying
  1. Origin
  2. Direction
  3. t-parameters (light ray distance)
- This information is passed to the RT core and is present in Ray Buffer under Ray Properties

Ray Generation in Nvidia Optix

```
extern "C" __global__ void __raygen__rg()
{
    // Lookup our location within the launch grid
    const uint3 idx = optixGetLaunchIndex();
    const uint3 dim = optixGetLaunchDimensions();

    // Map our launch idx to a screen location and create a ray from
    // the camera location through the screen
    float3 ray_origin, ray_direction;
    computeRay( idx, dim, ray_origin, ray_direction );
....
}
```

```
unsigned int p0, p1, p2;
optixTrace(
        params.handle,
        ray_origin, ray_direction,
        0.0f,   // Min intersection distance
        1e16f,  // Max intersection distance
        0.0f,   // ray-time -- used for motion blur
        OptixVisibilityMask( 255 ), // Specify always visible
        OPTIX_RAY_FLAG_NONE,
        0, 0, 0, // SBT stride -- See SBT discussion
        p0, p1, p2 ); // These 32b values are the ray payload

// Our results were packed into opaque 32b registers
float3 result; result.x = int_as_float( p0 ); result.y = int_as_float( p1 ); result.z = int_as_float( p2 );
// Record results in our output raster
params.image[idx.y * params.image_width + idx.x] = make_color( result );
```

BVH traversal (RT Core)

- traverseAS instruction is performed in RT core completely
- Inputs to this long latency function are:
    1. Ray information - origin, direction, and t-parameters
    2. Bounding Volume Hierarchy - BVH rootnode (Acceleration Structure)
- Output of this RT core based on BLAS leaf nodes are
    1. Two floating-point values representing the barycentric coordinates (u,v) of the hit point inside the triangle
    2. Hit information of BLAS leaf nodes in intersectionBuffer in case of procedural mesh
- worldToObjectMatrix is applied to ray to convert the ray coordinates to the BLAS's coordinate system (the TLAS have coordinates in World space while the BLAS objects have coordinates in Local object space)

```
Require: Ray, AccelerationStructure

while Ray Not Terminated do
    // Traverse inside TLAS tree
    while Node.Type == TopInternalNode do
        Traverse to the next node
    end while

    // Reached TLAS Leaf Node
    while Node.Type == TopLeafNode do
        // Apply the worldToObjectMatrix to the ray to transform it to the BLAS's coordinate system
        Apply worldToObjectMatrix to Ray

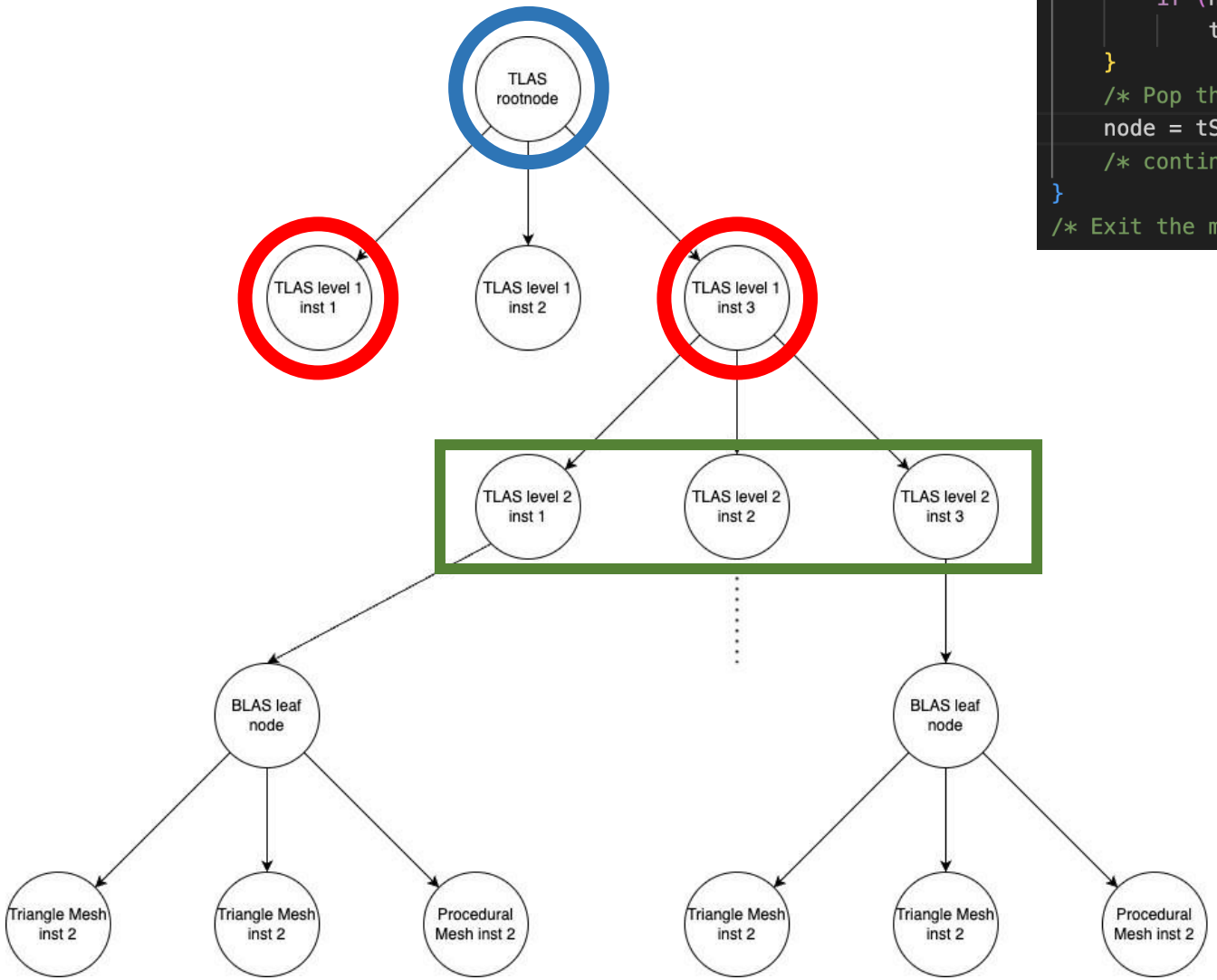
        // Traverse inside BLAS tree
        while Node.Level == BottomLevel do
            // Traverse through BLAS Internal node
            while Node.Type == BottomInternalNode do
                Traverse to the next node
            end while

            // Reached BLAS Leaf Node
            while Node.Type == BottomLeafNode do
                if Node.LeafType == Triangle then
                    hit = Perform primitive intersection test

                    if hit == true then
                        Update closest-hit geometry
                    endif
                else
                    /* Intersection shader for Procedural meshes is applied outside RT core */
                    add intersection to intersectionBuffer
                end if
            end while
        end while
    end while
end while
```

Traversal Stack example for ray 1

Traversal of TLAS rootnode children

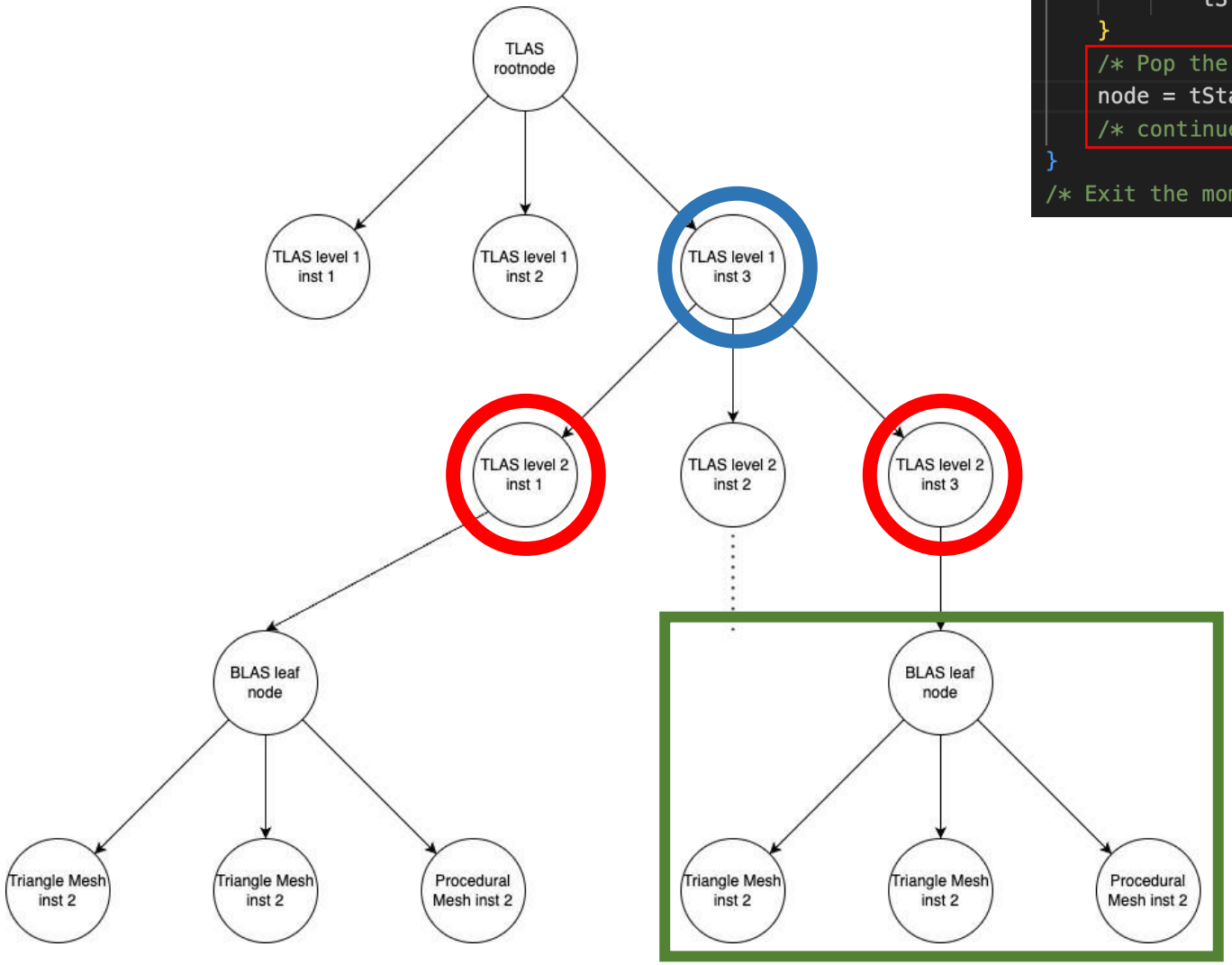



```
while (node != leafnode)
{
    /* Put all the TLAS internal nodes for a particular level of BVH in the traversal stack */
    foreach child in node
    {
        hitNode = RayBoxTest(ray, child.boundingBox)
        if (hitNode == true)
            tStack.push(child)
    }
    /* Pop the last pushed TLAS internal node from tStack */
    node = tStack.pop()
    /* continue finding the next level's TLAS internal node Bounding box intersection */
}
/* Exit the moment you encounter a leafnode (BLAS) */
```

Traversal Stack
TLAS level 1 inst 3
TLAS level 1 inst 1

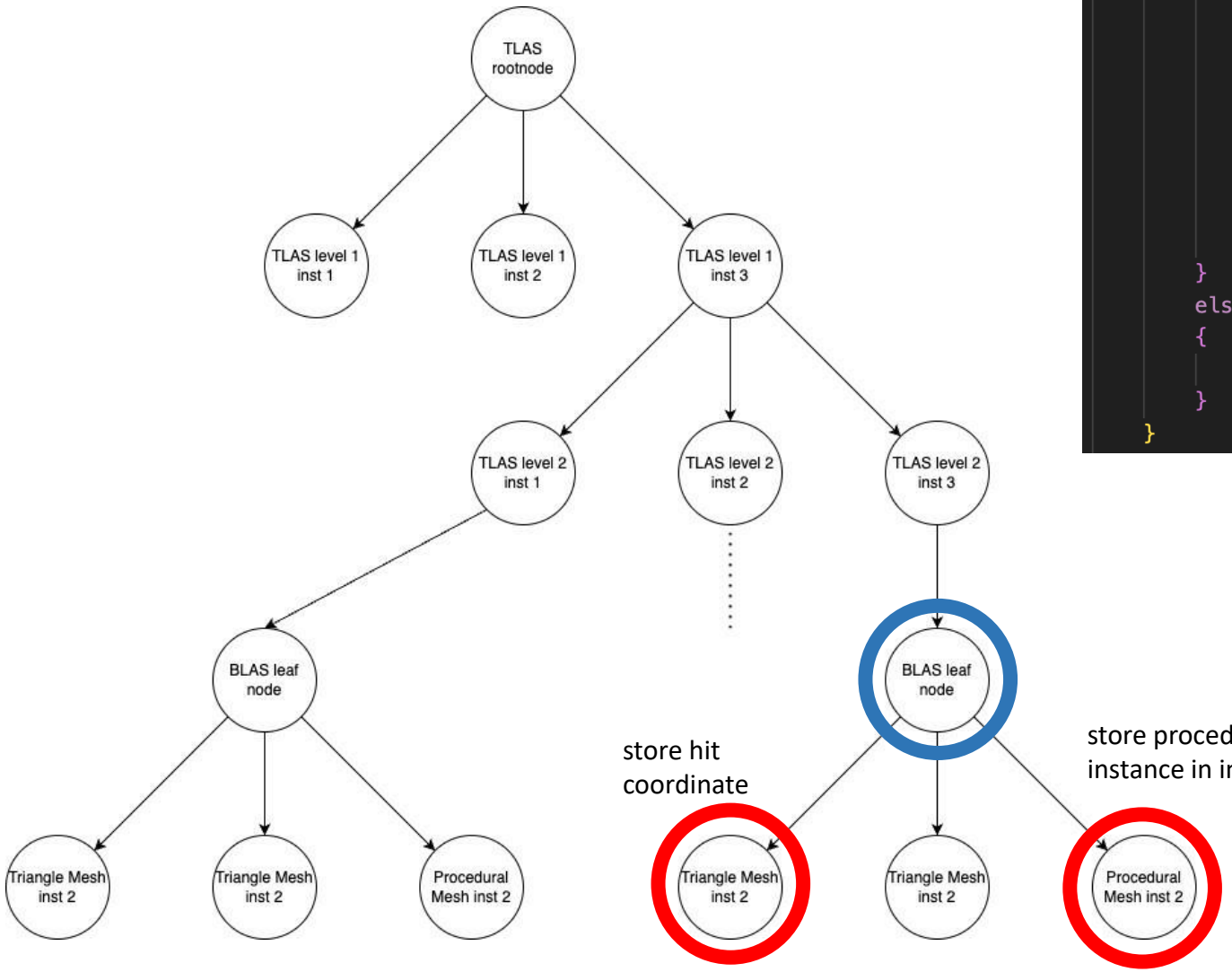
Traversal of TLAS level 1 inst 3 children

```
while (node != leafnode)
{
    /* Put all the TLAS internal nodes for a particular level of BVH in the traversal stack */
    foreach child in node
    {
        hitNode = RayBoxTest(ray, child.boundingBox)
        if (hitNode == true)
            tStack.push(child)
    }
    /* Pop the last pushed TLAS internal node from tStack */
    node = tStack.pop()
    /* continue finding the next level's TLAS internal node Bounding box intersection */
}
/* Exit the moment you encounter a leafnode (BLAS) */
```



Traversal Stack
TLAS level 2 inst 3
TLAS level 2 inst 1
TLAS level 1 inst 1

Traversal of TLAS level 2 inst3/BLAS leafnode children



store hit coordinate

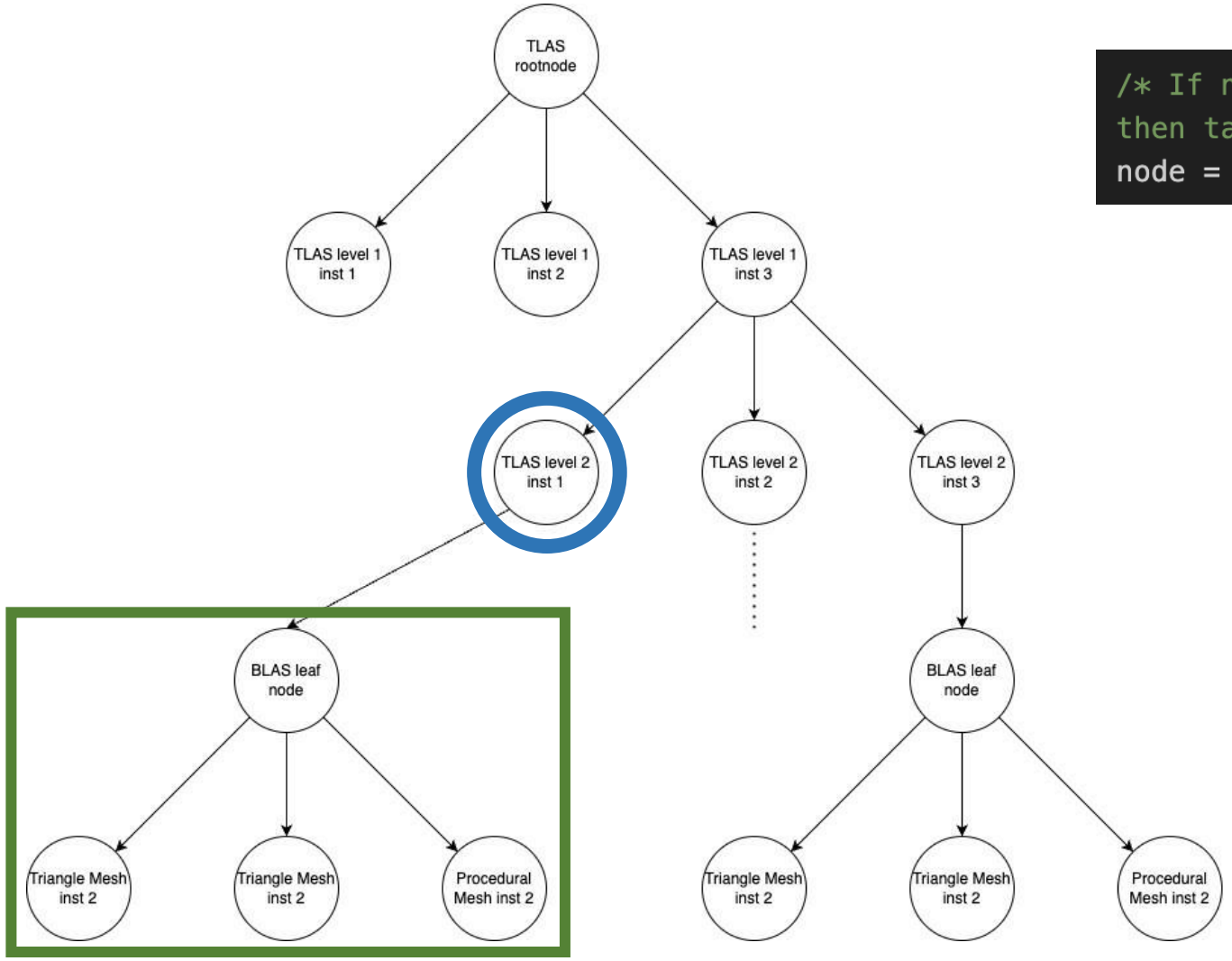
store procedural geometry instance in intersectionBuffer

```
/* After reaching TLAS leafnode (BLAS top level) in BVH,
check for Ray-triangle tests for that leafnode */
while (node == leafnode)
{
    foreach BLASleafnode in node
    {
        if (BLASleafnode == triangle)
        {
            new_hit_point_position = RayTriangleTest(ray, BLASleafnode)
            /* If any hit then stop */
            if (hit == true and anyhit == true)
                break

            /* Update closest hit information if the new hit is closer to the ray origin */
            if (new_hit_point_position < hit_position)
            {
                hit_position = new_hit_point_position
            }
        }
        else if (BLASleafnode == procedural)
        {
            intersectionBuffer.push(BLASleafnode)
        }
    }
```

Traversal Stack
TLAS level 2 inst 1
TLAS level 1 inst 1

Traversal of TLAS level 2 inst 1 children




```
/* If no hits are encountered in the current TLAS leafnode
then take the next TLAS internal node and continue the search */
node = tStack.pop()
```

Traversal Stack
TLAS level 1 inst 1

Applying Closet-hit Shaders based on BVH traversal (CUDA Core)

- After traverseAS() we need to perform Intersection shaderfor the Procedural mesh geometries.

- Once all hit geometries are determined, we apply Closest-hit shaders or Miss shaders to all the geometries hit by the ray.

```
traverseAS()
intersectionIdx = 0
// call all intersection shaders for the procedural leaf nodes in a loop from intersectionBuffer
// (Intersection shader for triangle leaf nodes are already applied inside traverseAS() function)
while intersectionExit(intersectionIdx) do
    // get Shader ID from Shader Binding table based on hitGroupID of the BLAS procedural node
    shaderID = getIntersectionShaderID()
    if shaderID == intersectionID0 then
        callIntersectionShader(shaderID)
    else if shaderID == intersectionID1 then
        callIntersectionShader(shaderID)
    end if
    intersectionIdx++
end while

// if geometry is intersected by the ray, call closest-hit shader of that object
if HitGeometry() then
    // get Shader ID from Shader Binding table based on hitGroupID of the geometry
    shaderID = getClosestHitShaderID()
    if shaderID == closestHitID0 then
        callClosestHitShader(shaderID)
    else if shaderID == closestHitID1 then
        callClosestHitShader(shaderID)
    end if
else
    // if ray misses object, call miss shader
    callMissShader()
end if

endTraceRay()
```

Closest Hit and Miss shaders in Nvidia Optix

```
extern "C" __global__ void __closesthit__ch()
{
    // When built-in triangle intersection is used, a number of
fundamental attributes are provided by the OptiX API,
including barycentric coordinates.
    const float2 barycentrics = optixGetTriangleBarycentrics();
    // Convert to color and assign to our payload outputs.
    const float3 c = make_float3( barycentrics, 1.0f );
    optixSetPayload_0( float_as_int( c.x ) );
    optixSetPayload_1( float_as_int( c.y ) );
    optixSetPayload_2( float_as_int( c.z ) );
}
```

```
extern "C" __global__ void __miss__ms()
{
    MissData* miss_data =
    reinterpret_cast<MissData*>( optixGetSbtDataPointer() );
    setPayload( miss_data->bg_color );
}
```

Ray Payload (maintained in CUDA core)

- The Ray Payload in Vulkan (rayPayloadInEXT/rayPayloadInEXT) contains user defined data which is initialized in Ray Generation shader
- The user defined variables are computed in Intersection shader in the CUDA core based on hit information from RT core
- These data values are then used for Closest-hit or Any-hit shader
- The Ray Payload can be defined to contain data values for
  1. Hit Point Position
  2. Surface Normals
  3. Texture Coordinates
  4. Material Properties

References and Important materials

- Vulkan-Sim: A GPU Architecture Simulator for Ray Tracing - Tor M. Aamodt
- Intersection Prediction for Accelerated GPU Ray Tracing - Tor M. Aamodt
- RT Engine: An Efficient Hardware Architecture for Ray Tracing
- Rays and Spheres: The MATH! // Ray Tracing series - The Cherno (YouTube)